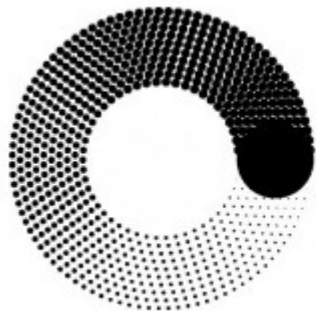




**МОСКОВСКИЙ
ПОЛИТЕХ**

**АВТОМАТИЗАЦИЯ ПРОЦЕССОВ ЖИЗНЕННОГО
ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ**

**Лабораторная работа № 3.
Контейнеризация**

**Москва
2024**

Теоретические основы

Docker

Docker — это программная платформа с открытым исходным кодом, которая помогает упростить процесс создания, управления, запуска и распространения приложений. С помощью Docker можно упаковать свое приложение вместе со всеми его зависимостями в контейнер. Контейнеры позволяют легко и единообразно развертывать ваши приложения.

Компании уже давно используют контейнерные технологии для устранения слабых мест виртуальных машин. Контейнеры в некотором смысле являются подобием облегченных версий виртуальных машин. Важное различие между контейнерами и виртуальными машинами заключается в том, что контейнерам не нужна собственная операционная система. Все контейнеры на узле совместно используют операционную систему хоста, что высвобождает много системных ресурсов.

То, что сейчас известно как современные контейнеры, начиналось как контейнеры Linux (LXC). Многие компании, в том числе Google, помогли внедрить технологии, связанные с контейнерами, в ядро Linux. Без этих вкладов у нас не было бы той богатой экосистемы контейнеров, которую мы имеем сегодня.

Контейнеризация была не очень доступна, пока не появился Docker. Контейнеры Docker создают абстракцию на уровне приложения. Контейнеры упаковывают приложение и его зависимости со всем необходимым для запуска, включая:

- операционную систему
- код приложения
- окружение
- системные инструменты
- системные библиотеки и другое

Образы

Образ — это шаблон с инструкциями по созданию контейнера Docker. Часто образ основано на другом образе с некоторой дополнительной настройкой. Например, вы можете создать образ на основе образа ubuntu, но установить веб-сервер Apache и ваше приложение, а также детали конфигурации, необходимые для запуска вашего приложения.

Вы можете создавать свои собственные образы или использовать только те, которые созданы другими и опубликованы в реестре. Чтобы создать собственный образ, вы создаете Dockerfile с простым синтаксисом для определения шагов, необходимых для создания образа и его запуска. Каждая инструкция в Dockerfile создает слой в образе. Когда вы меняете Dockerfile и перестраиваете образ, перестраиваются только те слои, которые изменились.

Это часть того, что делает образы такими легкими, маленькими и быстрыми по сравнению с другими технологиями виртуализации.

Контейнеры

Контейнер — это исполняемый экземпляр образа. Вы можете создавать, запускать, останавливать, перемещать или удалять контейнеры с помощью Docker API или CLI. Вы можете подключить контейнер к одной или нескольким сетям подключить к нему хранилище или даже создать новый образ на основе его текущего состояния.

По умолчанию контейнер относительно хорошо изолирован от других контейнеров и своего хост-компьютера. Вы можете контролировать, насколько изолированы сеть контейнера, хранилище или другие базовые подсистемы от других контейнеров или от хост-компьютера.

Контейнер определяется образом из которого он был запущен, а также любыми параметрами конфигурации, которые вы предоставляете ему при создании или запуске. При удалении контейнера любые изменения его состояния, не сохраненные в постоянном хранилище, исчезают.

Основные термины docker

Пространство имен: пространства имен создаются при запуске контейнера. Они обеспечивают уровень изоляции, поскольку каждый элемент контейнера работает в своем пространстве имен.

Cgroups: группы управления позволяют распределять ресурсы между процессами, работающими в системе.

Docker Engine: это основной продукт Docker, который включает в себя демон и интерфейс командной строки. У него есть API для взаимодействия с демоном Docker.

Dockerfile: текстовый документ, содержащий инструкции по созданию образов Docker.

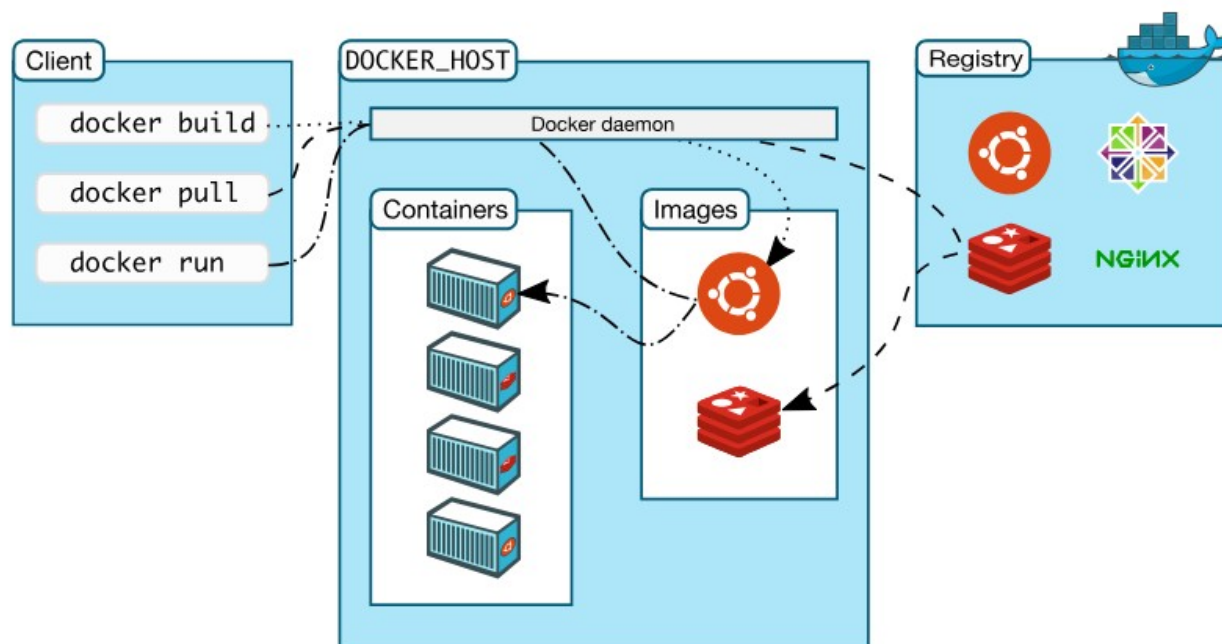
Docker registry (реестр): Реестр Docker позволяет хранить и распространять именованные образы Docker. Реестры организованы в репозитории и содержат все версии различных образов.

Docker Hub: сервис, публичный реестр Docker, который позволяет находить контейнеры и делиться ими.

Docker tag (теги). Теги Docker похожи на метки версии, которые вы можете присвоить любой завершенной сборке (образу).

Объединенная файловая система (AUFS): объединенная файловая система объединяет несколько каталогов на одном хосте и представляет их как один каталог.

Архитектура docker



Основные команды docker

docker pull — это команда, позволяющая загрузить определенный образ или набор образов.

docker run — команда запуска позволяет создать контейнер из указанного образа и запустить этот контейнер с помощью заданной команды.

docker build — это команда, которая используется для создания образа из Dockerfile.

docker stop — это команда, которая используется для остановки контейнера.

docker ps — команда просмотра запущенных контейнеров.

docker images — команда просмотра сохраненных образов

Задания (Часть 1)

1. Установить Docker на всех развернутых в ходе ЛР2 виртуальных машинах
2. На усмотрение студента выбрать Docker-образ, который будет являться основой для разрабатываемого приложения
3. Получить образ на TEST-виртуальную машину
4. Добавить в корень проекта файл Dockerfile, внутри которого происходит сборка и запуск проекта из исходных файлов
5. Собрать приложение внутри Docker, запустить контейнер из полученного образа

Docker Compose

Создать контейнер из образа довольно просто, выполнив одну команду. Docker-compose еще больше автоматизирует это.

Docker-compose вращается вокруг файла конфигурации с именем `docker-compose.yml`. В нем мы определяем все наши сервисы. Сервис это часть вашего приложения; база данных или API, например. Все сервисы полагаются на образы, с помощью которых мы создаем контейнеры. Запуск контейнера может иметь множество вариантов; то, как эти параметры настроены, будет храниться в файле `yml`.

Примером этих параметров запуска является сопоставление портов при запуске контейнера. При обычном использовании `docker` нам пришлось бы вызвать

```
docker run -p 5000:5000 python-docker
```

в нашем терминале для запуска условного сервиса на 5000 порту. Docker-compose позволяет нам определить эти параметры в файле. Еще одним преимуществом наличия всех наших сервисов со всеми соответствующими параметрами сборки в одном файле является то, что мы можем создавать и запускать все наши сервисы одновременно, вызывая

```
docker-compose up
```

Запуск наших контейнеров можно сделать более гибким с помощью переменных среды. Мы можем предоставить `docker-compose` файл, содержащий некоторые переменные среды. Таким образом, мы можем безопасно предоставлять пароли без их жесткого кодирования в файле конфигурации.

Пример: мы создадим файл с именем `.env`, в котором зададим

```
DBPASSWORD=secretpass
```

Затем мы можем использовать `DBPASSWORD` в качестве переменной в файле `docker-compose.yml`.

Для этого мы указываем наш файл `env` при вызове `docker-compose`:

```
docker-compose --env-file .env up
```

Это убережет наши пароли от попадания в репозитории через запись в `docker-compose.yml`. Кроме того, это добавляет больше гибкости и делает проект более аккуратным, поскольку наши пароли больше не жестко запрограммированы.

Общая контейнерная сеть

Когда мы запускаем наши контейнеры с помощью `docker-compose`, он определяет сеть, которую совместно используют все наши сервисы. Это означает, что все сервисы могут обмениваться данными внутри.

Проиллюстрируем это на примере. Представьте, что мы разместили наше приложение на сайте `polytech.mos`. Когда нашему API необходимо взаимодействовать с нашей базой данных, ему не нужно подключаться через `polytech.mos:5432`, а вместо этого можно просто вызвать базу данных изнутри сети `docker-compose`. Это положительно влияет на безопасность, потому что это означает, что только наши службы имеют доступ к нашей базе данных, клиенты не могут получить доступ к службе из-за пределов приложения.

Переносимость и контроль версий

Поскольку у нас есть вся наша конфигурация в одном файле, мы можем легко поделиться этим файлом через систему контроля версий, такую как `Git`. Пользователи могут просто извлечь файл `docker-compose.yml` и исходный код, и все контейнеры будут запущены! Дополнительным преимуществом является то, что мы храним полную историю всех изменений, которые мы вносим в нашу конфигурацию, чтобы мы могли восстановить предыдущую версию, когда захотим. Это также упрощает настройку конвейера `CI/CD` для этого приложения.

Гибкость

Так как все наши сервисы полностью изолированы друг от друга, мы можем легко добавить новый сервис. Возможно, в будущем нашему приложению понадобится кэширование, просто добавим контейнер `Redis`! Другие службы, такие как наш API, могут легко подключаться к новой службе `Redis` по внутренней сети.

Пример файла docker-compose.yml

```
version: '3'

services:

  mospoly_database: #Имя сервиса
    container_name: mospoly_database #Имя с которым запустится контейнер
    hostname: mospoly_database #Имя "хоста" которым будет считать себя контейнер
    image: postgres #Образ из которого запускается контейнер
    volumes:
#Подключаемые директории
      - ./volume:/var/lib/postgresql #Директории вида хост_директория:
директория_контейнера

    environment: #переменные окружения
      - POSTGRES_DB=mospolydb, mospolydb_dev
      - POSTGRES_USER=mospolyuser
      - POSTGRES_PASSWORD=averyStrongPassword
    ports: # Сопоставление портов хост-машины и контейнера
      - 54321:5432
    restart: unless-stopped #условие для рестарта в случае остановки контейнера

  mospoly_api:
    container_name: mospoly_api
    hostname: mospoly_api
    build: # Параметры для сборки приложения
      context: ./mospoly_api # Каталог, используемый для сборки
      dockerfile: Dockerfile # Файл, используемый для сборки образа приложения
    ports:
      - 54322:5000
    volumes:
      - ./mospoly_api/src:/usr/src/app/
      - /usr/src/app/node_modules
    restart: unless-stopped
    environment:
      NODE_ENV: ${MOSPOLY_ENVIRONMENT}
    depends_on: # Указание на зависимость сервиса от другого сервиса
      - mospoly_database

  mospoly_webserver:
    container_name: mospoly_webserver
    hostname: mospoly_webserver
    build:
      context: ./mospoly_webserver
      dockerfile: Dockerfile
    ports:
      - 80:80
      - 443:443
    volumes:
      - ./mospoly_webserver/src/test:/usr/share/nginx/html
    restart: always
    depends_on:
      - mospoly_database
```

Задания (Часть 2)

1. Установить на все ВМ утилиту docker-compose
2. На выбор студента подобрать подходящую для приложения СУБД и получить с Dockerhub ее официальный (либо сторонний) образ. В случае, если приложение не использует БД, выбрать любой сторонний внешний сервис, используемый приложением (Redis, Minio и др.)
3. В корне проекта создать файл docker-compose.yml, в котором описать параметры для сборки и запуска разрабатываемого приложения а также запуска стороннего сервиса, выбранного студентом. В конфигурации указать подключаемые директории (volume), в которых будут храниться файлы запущенных приложений (логи, файлы БД и другие).
4. Запустить приложение и продемонстрировать его работу со сторонним сервисом.
5. Внести изменения, требующие записи информации в файлы приложения (добавить записи в базу данных через приложение, создать файл и др.)
6. Остановить и запустить приложение еще раз, продемонстрировать, что изменения, внесенные в ходе работы, были сохранены.
7. Подготовить отчет по лабораторной работе (текстовый документ и скриншоты с описанием по вышеописанным пунктам).

Вопросы для изучения

На данные вопросы необходимо ответить при защите лабораторной работы.

1. Каким образом docker обеспечивает изоляцию контейнеров от хостовой ОС и друг от друга?
2. Какие параметры требуется передать контейнеру docker при запуске, чтобы его сетевой адаптер был тождественен сетевому адаптеру хостовой ОС и почему?
3. Какую последовательность команд требуется ввести, чтобы при помощи docker-compose сначала собрать приложение, затем запустить его в фоновом режиме и затем подключиться к его выводу stdout и stderr?